

理论重点回答

1. RadixAttention / RadixCache 解决什么问题

自回归推理中，prompt 的每个前缀 token 都要先经过一次 prefill，产生后续 decode 所需的 K/V 张量（KV cache）。多轮对话、few-shot、RAG、树状思考和 SGLang 的多次 gen/fork 往往共享前缀；若每次请求结束就丢弃 KV cache，下一次只能重复计算，浪费计算和显存。

SGLang 的 RadixAttention 把 KV cache 当作可复用的缓存：用 radix tree（压缩前缀树）保存“token 序列前缀 → KV cache 页”的映射，按最长匹配前缀复用，并配合 LRU 淘汰和 cache-aware 调度提高命中率。实现中的 RadixCache 可理解为这棵树及其缓存管理器。它解决的是“跨请求/跨调用识别、保存、共享和淘汰相同前缀 KV”的问题；它关注的是缓存内容的生命周期与复用策略，不是单纯的显存连续分配。

2. page-sized KV cache 与 prefix reuse 在推理流程中的位置

- page-sized KV cache（分页 KV）：属于 KV cache 的物理存储和访问。在 prefill 计算出 KV 后，系统把序列 KV 切成固定大小的 page/block，按需分配到不连续的 GPU 物理位置；decode 每生成一个 token 就写入当前页，注意力 kernel 通过映射表读取这些页。它贯穿 prefill 后的 KV 保存以及后续 decode 的读写。
- prefix reuse（前缀复用）：属于新请求到达时的逻辑匹配和计算跳过。系统先将输入 token 与已有缓存做最长公共前缀匹配；命中的前缀直接取得已有 KV，只对未命中的 suffix 做 prefill，然后进入 decode。因此它主要减少 prefill 的重复计算，也减少重复存储。

二者可以组合：prefix tree 决定“复用哪段前缀”，paged KV 决定“这段 KV 放在哪些页、如何读取”。SGLang 论文中 RadixAttention 的 KV 本身就采用 non-contiguous、按 token page 的布局。

3. vLLM 的 PagedAttention 解决什么问题

传统 serving 往往为每个请求预留一整段连续 KV 内存，按最大上下文长度估计，导致内部碎片（实际未用的预留槽位）和外部碎片（不同大小连续区难以拼接）；动态长度、并发 batch 和 beam/parallel sampling 还会加剧浪费。PagedAttention 借鉴操作系统虚拟内存，把每条序列的 KV 切成固定 token 数的 block，维护逻辑 block 到物理 block 的 block table；物理 block 可以不连续，按需增长，注意力 kernel 分块读取。因此它降低碎片、避免提前为最大长度保留空间，并支持 block 粒度共享与 copy-on-write。vLLM 是在此机制上配套调度和 KV 内存管理的 serving engine。

4. PagedAttention 与 SGLang RadixAttention、paged KV 的联系和区别

联系：三者都利用 KV cache，且都允许 KV 以分页/非连续方式存放；RadixAttention 可以兼容 paged attention。分页布局为共享 KV 提供了可管理的物理载体，减少显存浪费；共享前缀则减少重复 prefill。

区别：

1. PagedAttention (vLLM) 是 attention kernel + block-table/内存分配方案，核心目标是解决 KV 的碎片、动态增长和物理映射，默认重点是单请求生命周期内的高效 serving；它本身不等于按 token 前缀建立跨请求缓存树。
2. RadixAttention / RadixCache (SGLang) 是运行时的前缀复用协议和缓存策略，核心目标是跨请求、跨调用、跨分支寻找最长共享前缀，并做 LRU 与 cache-aware 调度；它可以建立在 paged KV 之上。
3. SGLang 的 paged KV 机制描述的是 RadixCache 中 KV 的存储粒度和地址形式（论文实现可按 token 分页）；它是 RadixAttention 的底层存储方式，不是另一种前缀匹配算法。vLLM 的 block 通常包含多个 token，页大小是工程折中：块越小碎片越少，块越大 kernel 并行和访存效率通常更好。

一句话概括：PagedAttention 管“KV 放哪里、怎么读写”，RadixAttention 管“哪些请求可以复用哪段 KV”；SGLang 把前缀树复用与分页 KV 结合起来，vLLM 主要以分页内存管理和高吞吐调度为核心。

复盘材料：见 作业感受.md 和 AI使用说明.md。